

# CS 3610 – SOFTWARE TESTING TOOLS

T.Y.B.Sc. Computer Science | Semester VI

## ■ Important Points & Notes for All Chapters

| Chapter | Topic                            | Page |
|---------|----------------------------------|------|
| 1       | Introduction to Test Case Design | –    |
| 2       | Test Cases for Simple Programs   | –    |
| 3       | Test Cases and Test Plan         | –    |
| 4       | Defect Report                    | –    |
| 5       | Testing Tools                    | –    |

## CHAPTER 1 – Introduction to Test Case Design

### 1.1 What is a Test Case?

- A **test case** describes input, action, and an expected result to determine if software functionality works correctly.
- IEEE defines test case as: *"a set of input values, execution pre-conditions, expected results and execution post-conditions, developed for a particular objective or test condition."*
- Test cases have to be designed based on two criteria: **Reliability** (detects all errors) and **Validity** (at least one test case reveals errors).
- A test case is associated with: Identifier, Name, Purpose, Required Inputs, Test Conditions, Expected Outputs.

### 1.2 Test Case Design Techniques

- Three broad categories:
  - **Specification-based (Black-box)**: Test cases derived from specification/model; does not use internal structure knowledge.
  - **Structure-based (White-box)**: Test cases derived from internal code structure; developers test code in detail.
  - **Experience-based**: Based on tester's skills, knowledge, and expertise with similar systems.

### 1.3 Errors, Bugs & Defects

| Term              | Definition                                                                      |
|-------------------|---------------------------------------------------------------------------------|
| <b>Error</b>      | Incorrect human action that produces an incorrect result (human mistake).       |
| <b>Bug/Defect</b> | Error, flaw, failure or fault in a program causing incorrect/unexpected result. |
| <b>Fault</b>      | Condition that leads to malfunctioning of software.                             |
| <b>Failure</b>    | State where software is unable to perform functions as per user requirements.   |

- Types of Errors: **Syntax Error** (missing/incorrect character in code) | **Logic Error** (incorrect program flow, e.g., infinite loop)
- Bug causes: **Wrong Coding** (wrong implementation), **Missing Coding** (feature not developed), **Extra Coding** (unwanted extra feature added)

## 1.4 Bug Tracking Tools

| Tool            | Key Features                                                                                                  |
|-----------------|---------------------------------------------------------------------------------------------------------------|
| <b>Jira</b>     | Open-source; bug tracking, project management, issue tracking; supports GitHub integration; custom workflows. |
| <b>Bugzilla</b> | Open-source; supports Linux/Windows/Mac; email notification; advanced search; time tracking.                  |
| <b>Redmine</b>  | Open-source; written in Ruby; supports MySQL, MS SQL, SQLite; role-based access control.                      |
| <b>MantisBT</b> | Stands for Mantis Bug Tracker; web-based; PHP; full-text search; revision control integration.                |
| <b>Backlog</b>  | IT project management; Gantt & burn down charts; supports Git/SVN; iOS/Android apps.                          |
| <b>BugNet</b>   | ASP.NET & C#; Microsoft SQL; email notification; milestone management.                                        |
| <b>Trac</b>     | Open-source; Python; supports Windows/Mac/Linux; multiple projects; wide range of plugins.                    |

## 1.5 Entry and Exit Criteria

- **Entry Criteria:** Conditions that must be satisfied BEFORE testing can begin (pre-conditions).
- **Exit Criteria:** Conditions that must be satisfied BEFORE testing can be STOPPED (post-conditions).
- Example: Entry – file xyz.dat should NOT exist; Exit – program should terminate without errors and file should be present.
- Entry criteria questions: Is documentation available? Is system ready? Is environment set up? Is quality level appropriate?

## 1.6 Test Cases in MS Excel (Standard Format)

| Field                   | Description                                              |
|-------------------------|----------------------------------------------------------|
| <b>Test Case ID</b>     | Unique ID (e.g., TC_UI_01 = User Interface Test Case #1) |
| <b>Test Case Name</b>   | Feature being tested, requirement number                 |
| <b>Test Description</b> | What and which condition is being tested                 |
| <b>Pre-conditions</b>   | Must be fulfilled before test case execution             |
| <b>Execution Steps</b>  | Steps to perform on system under test                    |
| <b>Expected Result</b>  | Desired output from execution steps                      |
| <b>Actual Result</b>    | Real result after execution                              |
| <b>Status</b>           | Pass (Green) / Fail (Red)                                |
| <b>Comments</b>         | Additional info or reasons                               |

## 1.7 Features of a Testing Method

- **High Probability of Detecting Errors:** Understand software thoroughly; find possible failure ways.
- **No Redundancy:** Every test should have a distinct purpose; avoid tests with same intended purpose.
- **Choose Most Appropriate Test:** Select tests likely to find more errors.
- **Moderate:** Tests should be neither too simple nor too complex; perform each test separately.

## CHAPTER 2 – Test Cases for Simple Programs

### 2.1 White-Box Testing

- **White-box testing** (also: clear box / glass box / open box) tests the external functionality by examining and testing program code.
- Takes into account: program code, code structure, and internal design flow.
- Two types: **Static testing** (only source code, no executables) and **Structural testing** (code, structure, internal design).
- **Code Coverage** = (Lines of code exercised / Total lines of code) × 100

### 2.2 Statement Coverage Testing

- Simplest form of white-box testing; each statement is executed **at least once**.
- Covers all possible paths, lines, and statements in source code.
- Cannot test false conditions; does not check loop termination; does not understand logical operators.

$$\text{Statement Coverage} = (\text{Number of executed statements} / \text{Total statements}) \times 100$$

### 2.3 Decision Coverage Testing

- Reports the **True or False** outcomes of each Boolean expression.
- Goal: cover and validate all accessible source code by ensuring each branch of every decision is executed at least once.
- Uses a **Control Flow Graph** to cover all possible outcomes.
- Eliminates problems that occur with statement coverage testing.

$$\text{Decision Coverage} = (\text{Decision outcomes exercised} / \text{Total decision outcomes}) \times 100$$

### 2.4 Branch Coverage Testing

- Every outcome from a code module (statement or loop) is tested.
- Covers **both conditional and unconditional branches** (Decision coverage covers only conditional).
- Considered minimum level for most software products.
- Like decision coverage, uses a Control Flow Graph.

$$\text{Branch Coverage} = (\text{Number of executed branches} / \text{Total branches}) \times 100$$

### 2.5 Loop Coverage Testing

- Validates loops; checks whether loop body executes zero times, exactly once, or more than once.
- **Goals:** Fix infinite loop problem; know performance; identify initialization problems; determine uninitialized variables.
- **Types of Loops:**
  - **Simple Loop:** Normal for/while/do-while loop.
  - **Nested Loop:** Loop inside another loop.
  - **Concatenated Loop:** Loop after another loop (loop is after loop).
  - **Unstructured Loop:** Combination of concatenated and nested loops.
- For loop testing, run tests: zero iterations, negative count, once, twice, n times, max-1 times, max times, max+1 times.

### 2.6 Path Coverage Testing

- Design test cases such that all **linearly independent paths** are executed at least once.
- A **linearly independent path** introduces at least one new edge not included in any other path.
- Uses **Control Flow Graph (CFG)**; describes how control flows through a program.

$$\text{Path Coverage} = (\text{Total paths exercised} / \text{Total paths in program}) \times 100$$

## 2.7 McCabe's Cyclomatic Complexity

- Measures complexity of a software program; quantitative measure of independent paths. Developed by Thomas J. McCabe in 1976.
- Three methods (all give same result):
  - **Method 1:**  $V(G) = E - N + 2$  (E = edges, N = nodes in CFG)
  - **Method 2:**  $V(G) = \text{Total bounded areas} + 1$
  - **Method 3:**  $V(G) = \text{Number of decision statements} + 1$

*Note: 100% branch coverage does NOT guarantee 100% path coverage. But 100% path coverage guarantees 100% branch and statement coverage.*

## CHAPTER 3 – Test Cases and Test Plan

### 3.1 Test Plan – Overview

- IEEE defines test plan as: *"a document describing the scope, approach, resources and schedule of intended testing activities."*
- A test plan is a pre-planned blueprint for the complete, successful testing process.
- It outlines: test strategy, objectives, timetable, estimation, deliverables, and resources.

### 3.2 Steps to Create a Test Plan

1. **Analyze the Product:** Deep understanding of product/feature; talk to designers/developers; review SOW, proposals.
2. **Design Test Strategy:** Define project objectives, effort, cost; scope, type of testing, risks, test logistics.
3. **Define Objectives:** List all software features to test; define ideal benchmark results.
4. **Establish Test Criteria:** Suspension Criteria (when to stop) and Exit Criteria (completion benchmarks).
5. **Plan Resource Allocation:** Human effort, equipment, infrastructure needed.
6. **Setup Test Environment:** Software and hardware setup for running tests; prefer real devices.
7. **Determine Schedule:** Break into tasks; allocate time/effort; consider risks and deadlines.
8. **Establish Test Deliverables:** Before testing: Test Plan, Design. During: Scripts, Test Data, Logs. After: Results, Defect Reports.

### 3.3 Test Plan Template Fields

- Test Plan Identifier | Introduction | Purpose | Objectives and Tasks
- Scope | Testing Strategy | Test Items | Software Risk Issues
- Features to be tested | Features NOT to be tested | Approach
- Item Pass/Fail Criteria | Suspension & Resumption Criteria | Test Deliverables
- Environmental Needs | Staffing & Training Needs | Responsibilities | Schedule
- Planning Risks & Contingencies | Approvals | Glossary

### 3.4 Writing Test Cases – Steps

1. Review the Requirements
2. Write a Test Plan
3. Identify the Test Suite
4. Name the Test Cases
5. Write Test Case Descriptions & Objectives
6. Create the Test Cases (write steps & specify test data)
7. Review the Test Cases

### 3.5 Test Report

- A test report summarizes all test activities and final test results.
- Used by product managers, analysts, testing team, and developers.
- Two types: **Test Incident Report** and **Test Summary Report**.
- **Test Incident Report (IEEE 829):** Records software incidents/defects; unique ID per incident; sections include: Incident Summary, Description (Inputs, Expected, Actual, Anomalies, Date/Time, Environment), Impact,

Investigation, Metrics, Disposition.

- **Test Summary Report (IEEE 829):** Summarizes completion of test activities; sections: Identifier, Summary, Variances, Comprehensive Assessment, Summary of Results (Resolved/Unresolved), Evaluation, Recommendations, Summary of Activities, Approvals.

## CHAPTER 4 – Defect Report

### 4.1 What is a Defect?

- A defect = variation between expected and actual result. Also called error, flaw, failure, or fault.
- **Root causes:** Requirements not clearly defined; wrong designs; untrained people; incapable processes.

### 4.2 Defect Life Cycle (Bug Life Cycle)

- Starts when a new defect is found; ends when tester closes it (won't recur).

| State          | Description                                                                             |
|----------------|-----------------------------------------------------------------------------------------|
| New            | Newly found defect; testing and validation performed.                                   |
| Assigned       | Assigned to developer by project lead/manager.                                          |
| Open           | Developer analyzes and works on fix. May move to Duplicate/Deferred/Rejected/Not a Bug. |
| Fixed          | Developer has made required changes to fix defect.                                      |
| Pending Retest | Waiting for tester to retest after developer fix.                                       |
| Retest         | Tester retesting to verify fix is correct.                                              |
| Reopen         | Issue still persists; assigned back to developer.                                       |
| Verified       | Tester confirms defect is fixed accurately.                                             |
| Closed         | Defect no longer exists.                                                                |
| Rejected       | Developer: defect not genuine.                                                          |
| Duplicate      | Same defect already reported elsewhere.                                                 |
| Deferred       | Low priority; will be fixed in a future release.                                        |
| Not a Bug      | Does not affect application functionality.                                              |

### 4.3 Defect Classification

- **Requirement Defects:** Functional defects (functionality absent/present as per customer), Interface defects (user interface, connectivity issues).
- **Design Defects:** Algorithmic defects (wrong decision designs), Module interface defects (communication between modules), User interface defects (navigation, look & feel), System interface defects (communication with environment).
- **Coding Defects:** Variable initialization defects, Commenting/documentation defects, Database-related defects.
- **Testing Defects:** Test design defects (in test plans/cases/data), Test tool defects, Test environment defects (wrong hardware/software setup).

### 4.4 Defect Severity vs Priority

| Severity      | Description                                     | Priority    | Description                                     |
|---------------|-------------------------------------------------|-------------|-------------------------------------------------|
| S1 – Critical | No workaround; complete failure.                | P1 – High   | Must fix in upcoming build; include in release. |
| S2 – Major    | Has workaround but difficult.                   | P2 – Medium | May fix after release / in next release.        |
| S3 – Minor    | Has easy workaround.                            | P3 – Low    | May or may not fix at all.                      |
| S4 – Trivial  | No impact on functionality; just inconvenience. |             |                                                 |

#### 4.5 Defect Report – Key Fields

- Defect ID (unique), Defect Name, Project Name, Module Name
- Phase Introduced / Phase Found, Defect Type, Severity (S1–S4), Priority (P1–P3)
- Summary, Description, Status (Open/Resolved/Closed/Deferred/Reopened)
- Reported By / Reported On, Version, Date Raised, Detected By, Fixed By, Date Closed
- Good defect report: Sufficient info to reproduce & fix; simple; helps stakeholders decide which defects to fix.

## CHAPTER 5 – Testing Tools

### 5.1 Manual Testing vs Automation Testing

| Manual Testing                                     | Automation Testing                                      |
|----------------------------------------------------|---------------------------------------------------------|
| Executed manually by human, without tools/scripts. | Test cases executed using automation tools/scripts.     |
| Oldest and most rigorous type.                     | Uses specialized tools to control test execution.       |
| Slow, costly, not scalable, inconsistent.          | Faster, reliable, repeatable, reusable, cost-effective. |
| Hard to execute on multiple platforms.             | Supports multiple platforms simultaneously.             |

### 5.2 Benefits of Automation Testing

- Faster Feedback Cycle
- Increased Efficiency
- Reliable (same operation each time)
- Repeatable
- Programmable (sophisticated tests)
- Comprehensive
- Reusable across versions
- Better Quality Software
- Fast execution
- Cost Reduction
- Improved Accuracy (100% accurate)
- Higher Test Coverage

### 5.3 Types of Test Tools

- **Static Test Tools:** Do not involve actual execution. Include coverage analyzers, flow analyzers, flow tests.
- **Dynamic Test Tools:** Test with 'live' data. Include mutation analyzers, emulators, test drivers.

| Tool Type                   | Used For                                       | Used By          |
|-----------------------------|------------------------------------------------|------------------|
| Test Management             | Managing, scheduling, defect logging, tracking | Testers          |
| Configuration Management    | Implementation, execution, tracking changes    | All Team         |
| Static Analysis             | Static Testing                                 | Developers       |
| Test Data Preparation       | Analysis, design, test data generation         | Testers          |
| Test Execution              | Implementation, Execution                      | Testers          |
| Test Comparators            | Comparing expected & actual results            | All Team         |
| Coverage Measurement        | Structural coverage                            | Developers       |
| Performance Testing         | Monitoring performance, response time          | Testers          |
| Project Planning & Tracking | Planning                                       | Project Managers |
| Incident Management         | Managing tests                                 | Testers          |

### 5.4 Test Automation Frameworks

- **Data Driven Framework:** Test data and scripts are separated; data stored in external files (Excel, CSV, DB). Same script runs with multiple data sets.
- **Keyword Driven Framework:** Test logic divided into keywords/functions. Examples: HP QTP, Selenium.
- **Modular Framework:** Application divided into modules tested independently. Requires programming knowledge; uses OOP concepts.
- **Hybrid Framework:** Combination of more than one framework; weaknesses of one compensated by strengths of another.

## 5.5 Popular Open Source Automation Tools

| Tool                  | Description                                                                      |
|-----------------------|----------------------------------------------------------------------------------|
| <b>Selenium</b>       | Most common; supports Java, C#, Python; multi-OS; includes IDE, WebDriver, Grid. |
| <b>Katalon Studio</b> | Tests Web, Mobile, Desktop APIs; built-in keywords; project templates.           |
| <b>Appium</b>         | Mobile apps; iOS & Android; client/server architecture; easy install.            |
| <b>Cucumber</b>       | BDD-based; acceptance testing; supports Ruby, Java, .NET.                        |
| <b>Apache JMeter</b>  | Load testing for web apps; also unit & functional testing; Java-based.           |
| <b>Robotium</b>       | Android UI testing; system/functional/acceptance testing.                        |
| <b>WATiR</b>          | Web Application Testing in Ruby; lightweight; readable tests.                    |
| <b>Sikuli</b>         | Image recognition-based; automates anything on screen; good for desktop apps.    |
| <b>TestNG</b>         | Supports unit, functional, integration, data-driven, end-to-end testing.         |
| <b>SoapUI</b>         | API testing for SOAP & REST; functional, performance, data-driven testing.       |
| <b>k6</b>             | Load & performance testing; cloud-native apps, APIs, microservices.              |

## 5.6 Selenium – Key Points

- Created by Jason Huggins in 2004. Supports Java, C#, Python, Perl, Ruby.
- Supports OS: Windows, Linux, Solaris, Macintosh; mobile: iOS, Android, Windows Mobile.
- Supports browsers: Chrome, Firefox, IE, Safari.
- **Components:** Selenium IDE (record/playback; Chrome & Firefox plugin), WebDriver (programming interface for tests), Selenium Grid (parallel testing on multiple machines).
- Selenium IDE generates code in C#, Java, Python, Ruby, and Selenese.
- **Installation:** Install Java JDK → Install Eclipse → Download Selenium zip (.jar files) → Configure project build path.

## 5.7 Steps for Automation Testing with Selenium

1. Create a Selenium WebDriver instance.
2. Configure browser if required.
3. Navigate to the required web page.
4. Locate the relevant web element.
5. Perform action on the web element.
6. Verify and validate the action.

## QUICK REVISION – Formulas & Definitions

| Topic                        | Key Formula / Definition                                                                |
|------------------------------|-----------------------------------------------------------------------------------------|
| <b>Statement Coverage</b>    | $SC = (\text{Executed statements} / \text{Total statements}) \times 100$                |
| <b>Decision Coverage</b>     | $DC = (\text{Decision outcomes exercised} / \text{Total decision outcomes}) \times 100$ |
| <b>Branch Coverage</b>       | $BC = (\text{Executed branches} / \text{Total branches}) \times 100$                    |
| <b>Path Coverage</b>         | $PC = (\text{Total paths exercised} / \text{Total paths}) \times 100$                   |
| <b>Code Coverage</b>         | $CC = (\text{Lines of code exercised} / \text{Total lines}) \times 100$                 |
| <b>Cyclomatic Complexity</b> | $V(G) = E - N + 2$ OR Bounded areas + 1 OR Decisions + 1                                |
| <b>Test Case (IEEE)</b>      | Set of input values, pre-conditions, expected results, and post-conditions.             |
| <b>Entry Criteria</b>        | Conditions that MUST be met BEFORE testing starts.                                      |
| <b>Exit Criteria</b>         | Conditions that MUST be met BEFORE testing is stopped.                                  |
| <b>Defect</b>                | Variation between expected and actual result.                                           |
| <b>S1 Severity</b>           | Critical – No workaround; complete failure.                                             |
| <b>S2 Severity</b>           | Major – Workaround exists but difficult.                                                |
| <b>S3 Severity</b>           | Minor – Easy workaround available.                                                      |
| <b>S4 Severity</b>           | Trivial – No functional impact.                                                         |
| <b>P1 Priority</b>           | High – Must fix in upcoming build.                                                      |
| <b>P2 Priority</b>           | Medium – May fix after release.                                                         |
| <b>P3 Priority</b>           | Low – May or may not fix.                                                               |